



Oasis Infobyte Internship Program

Data Analytics

Task 1: Exploratory Data Analysis (EDA) on Retail Sales Data

Prepared by: Jahangeer Hassan

Internship Duration: July – August 2026

Objective

The objective of this project is to perform Exploratory Data Analysis (EDA) on a retail supermarket sales dataset to identify customer purchasing patterns, sales trends, product performance, and business insights using Python.

Step 2 (Import Libraries)

```
In [1]: import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline

plt.style.use("ggplot")
sns.set_theme(style="whitegrid")
```

Step 3 (Load Dataset)

```
In [4]: import pandas as pd

df = pd.read_csv("/content/supermarket.csv")

df.head()
```

Out[4]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment
0	1	CA-2017-152156	08/11/2017	11/11/2017	Second Class	CG-12520	Claire Gute	Consumer
1	2	CA-2017-152156	08/11/2017	11/11/2017	Second Class	CG-12520	Claire Gute	Consumer
2	3	CA-2017-138688	12/06/2017	16/06/2017	Second Class	DV-13045	Darrin Van Huff	Corporate
3	4	US-2016-108966	11/10/2016	18/10/2016	Standard Class	SO-20335	Sean O'Donnell	Consumer
4	5	US-2016-108966	11/10/2016	18/10/2016	Standard Class	SO-20335	Sean O'Donnell	Consumer

Step 4 – Dataset Overview

```
In [6]: print("Dataset Shape:")
print(df.shape)

print("\nColumn Names:")
print(df.columns.tolist())

print("\nDataset Information:")
df.info()
```

Dataset Shape:
(9800, 18)

Column Names:
['Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode', 'Customer ID', 'Customer Name', 'Segment', 'Country', 'City', 'State', 'Postal Code', 'Region', 'Product ID', 'Category', 'Sub-Category', 'Product Name', 'Sales']

iDataset Information:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 9800 entries, 0 to 9799

Data columns (total 18 columns):

#	Column	Non-Null Count	Dtype
0	Row ID	9800 non-null	int64
1	Order ID	9800 non-null	object
2	Order Date	9800 non-null	object
3	Ship Date	9800 non-null	object
4	Ship Mode	9800 non-null	object
5	Customer ID	9800 non-null	object
6	Customer Name	9800 non-null	object
7	Segment	9800 non-null	object
8	Country	9800 non-null	object
9	City	9800 non-null	object
10	State	9800 non-null	object
11	Postal Code	9789 non-null	float64
12	Region	9800 non-null	object
13	Product ID	9800 non-null	object
14	Category	9800 non-null	object
15	Sub-Category	9800 non-null	object
16	Product Name	9800 non-null	object
17	Sales	9800 non-null	float64

dtypes: float64(2), int64(1), object(15)

memory usage: 1.3+ MB

Observation

- The dataset contains retail sales transaction records.
- It includes customer information, product details, shipping information, sales, profit, discount, and quantity.
- These variables will be used to identify sales trends and generate business insights.

Step 5 – Data Quality Assessment

```
In [7]: print(" Missing Values")
display(df.isnull().sum())

print("\n Duplicate Records")
```

```
print(df.duplicated().sum())
```

Missing Values

	0
Row ID	0
Order ID	0
Order Date	0
Ship Date	0
Ship Mode	0
Customer ID	0
Customer Name	0
Segment	0
Country	0
City	0
State	0
Postal Code	11
Region	0
Product ID	0
Category	0
Sub-Category	0
Product Name	0
Sales	0

dtype: int64

Duplicate Records

0

Observation

- The dataset was checked for missing values and duplicate records.
- Missing values, if present, will be handled before analysis.
- Duplicate records can affect analysis accuracy and should be removed if found.

Step 6 – Data Cleaning

```
In [8]: # Remove duplicate rows
df = df.drop_duplicates()

# Fill missing Postal Code values (if any)
if "Postal Code" in df.columns:
    df["Postal Code"] = df["Postal Code"].fillna(df["Postal Code"].median())

print("Remaining Missing Values:")
display(df.isnull().sum())
```

Remaining Missing Values:

0

Row ID	0
Order ID	0
Order Date	0
Ship Date	0
Ship Mode	0
Customer ID	0
Customer Name	0
Segment	0
Country	0
City	0
State	0
Postal Code	0
Region	0
Product ID	0
Category	0
Sub-Category	0
Product Name	0
Sales	0

dtype: int64

Observation

Data cleaning improves the quality of the dataset by removing duplicate records and handling missing values. This ensures that the analysis is based on reliable and consistent data.

Step 7 – Descriptive Statistics

```
In [9]: # Descriptive Statistics  
df.describe()
```

```
Out[9]:
```

	Row ID	Postal Code	Sales
count	9800.000000	9800.000000	9800.000000
mean	4900.500000	55276.498571	230.769059
std	2829.160653	32023.374393	626.651875
min	1.000000	1040.000000	0.444000
25%	2450.750000	23223.000000	17.248000
50%	4900.500000	58103.000000	54.490000
75%	7350.250000	90008.000000	210.605000
max	9800.000000	99301.000000	22638.480000

Descriptive Statistics

The `describe()` function provides a statistical summary of all numerical columns in the dataset, including count, mean, standard deviation, minimum, maximum, and quartile values. This helps understand the distribution and variability of the data.

Step 8 – Mean

```
In [10]: # Mean of Numerical Columns  
df.mean(numeric_only=True)
```

Out[10]:

	0
Row ID	4900.500000
Postal Code	55276.498571
Sales	230.769059

dtype: float64

Observation

The mean represents the average value of each numerical feature and provides an overview of the central tendency of the dataset.

Step 9 – Median

```
In [11]: # Median of Numerical Columns  
df.median(numeric_only=True)
```

Out[11]:

	0
Row ID	4900.50
Postal Code	58103.00
Sales	54.49

dtype: float64

Observation

The median is the middle value of each numerical feature and is less affected by outliers than the mean.

Step 10 – Mode

```
In [12]: # Mode of Numerical Columns  
df.mode(numeric_only=True)
```

Out[12]:

	Row ID	Postal Code	Sales
0	1	10035.0	12.96
1	2	NaN	NaN
2	3	NaN	NaN
3	4	NaN	NaN
4	5	NaN	NaN
...	...	...	...
9795	9796	NaN	NaN
9796	9797	NaN	NaN
9797	9798	NaN	NaN
9798	9799	NaN	NaN
9799	9800	NaN	NaN

9800 rows × 3 columns

Observation

The mode represents the most frequently occurring value in each numerical column.

Step 11 – Standard Deviation

```
In [14]: # Standard Deviation
df.std(numeric_only=True)
```

Out[14]:

	0
Row ID	2829.160653
Postal Code	32023.374393
Sales	626.651875

dtype: float64

Observation

Standard deviation measures the spread of the data. A higher value indicates greater variability in the dataset.

Step 12 – Skewness

```
In [15]: # Skewness  
df.skew(numeric_only=True)
```

```
Out[15]: 0
```

Row ID	0.000000
Postal Code	-0.131663
Sales	12.983483

dtype: float64

Observation

Skewness indicates whether the distribution is symmetric or skewed. Positive values indicate right-skewed distributions, while negative values indicate left-skewed distributions.

Step 13 – Kurtosis

```
In [16]: # Kurtosis  
df.kurt(numeric_only=True)
```

```
Out[16]: 0
```

Row ID	-1.200000
Postal Code	-1.490955
Sales	304.445088

dtype: float64

Observation

Kurtosis measures the peakedness of the distribution. High kurtosis indicates more extreme values (outliers) compared to a normal distribution.

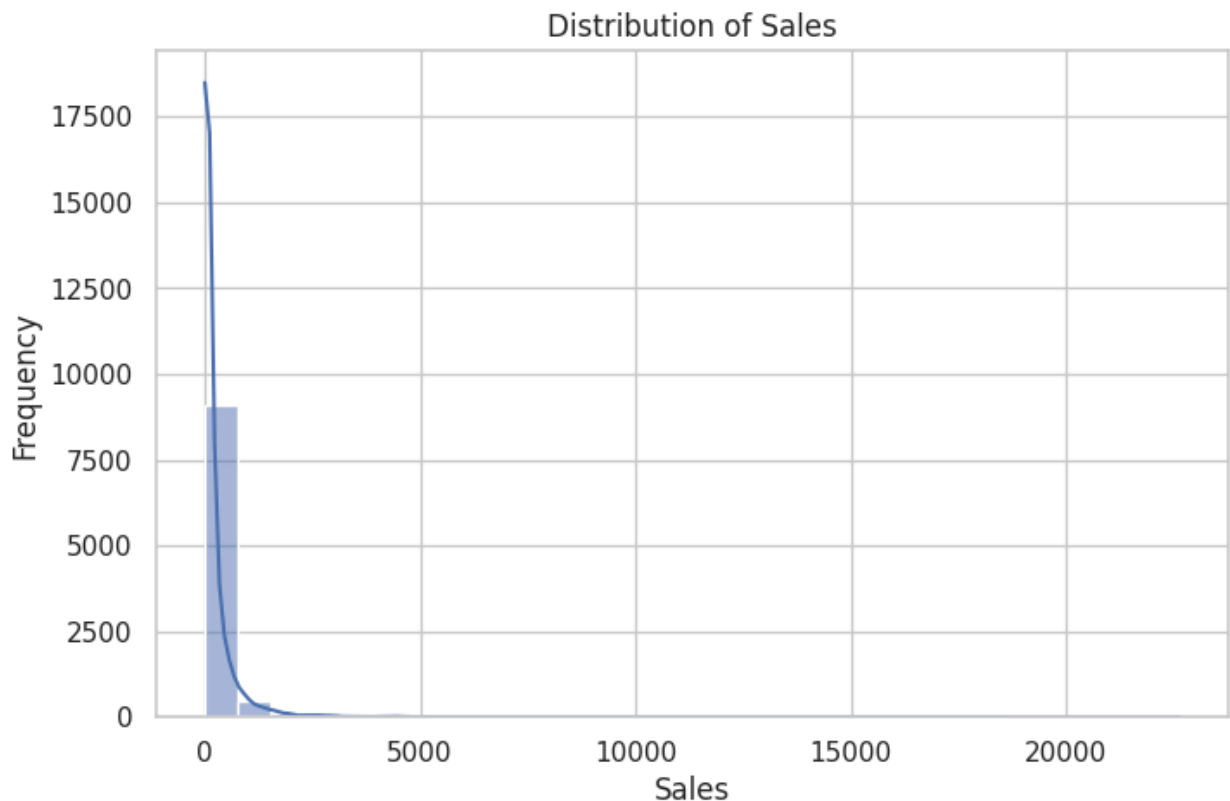
Step 14 – Sales Distribution

```
In [17]: plt.figure(figsize=(8,5))

sns.histplot(df["Sales"], bins=30, kde=True)

plt.title("Distribution of Sales")
plt.xlabel("Sales")
plt.ylabel("Frequency")

plt.show()
```



Observation

- Most sales transactions are of lower values.
- A few transactions have very high sales, indicating the presence of outliers.
- The sales distribution is positively skewed.

Step 15 – Category-wise Sales

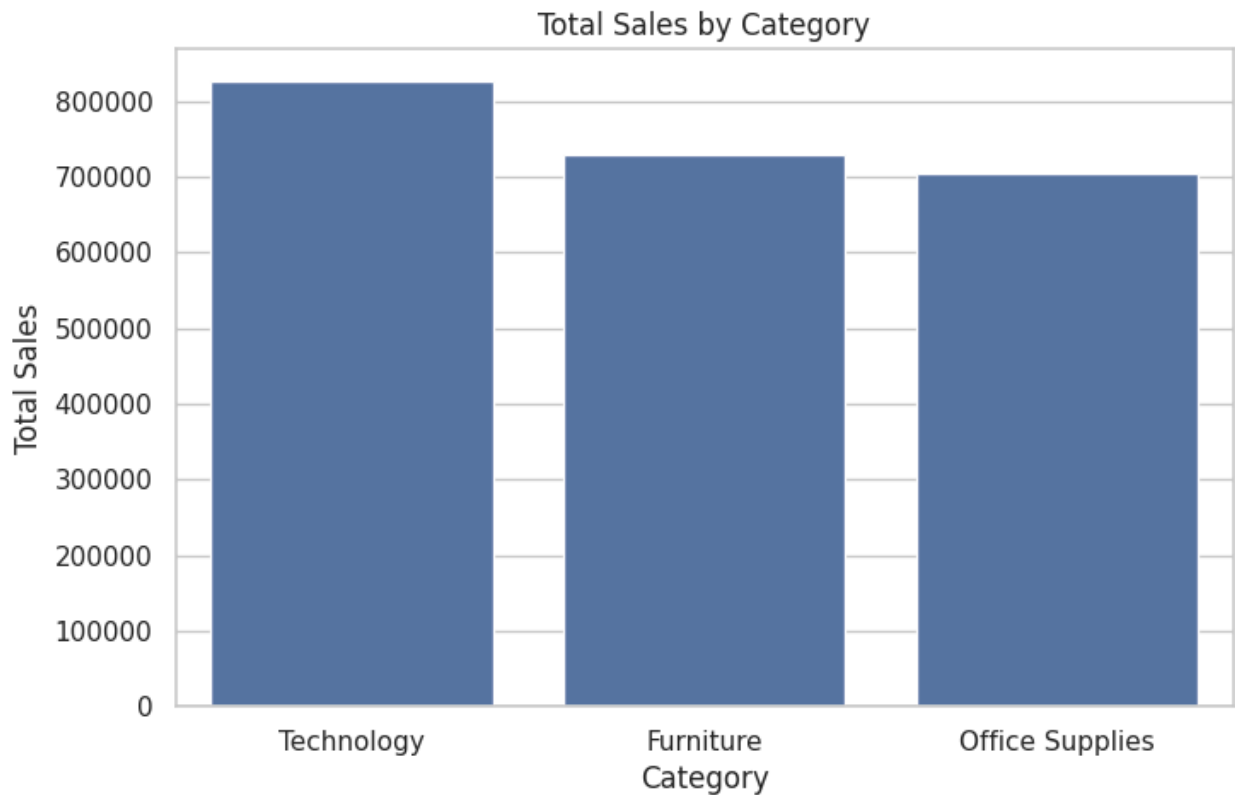
```
In [18]: plt.figure(figsize=(8,5))
```

```
category_sales = df.groupby("Category")["Sales"].sum().sort_values(ascending=False)

sns.barplot(x=category_sales.index,
            y=category_sales.values)

plt.title("Total Sales by Category")
plt.xlabel("Category")
plt.ylabel("Total Sales")

plt.show()
```



Observation

- Technology generated the highest sales.
- Office Supplies and Furniture contributed a significant portion of total revenue.
- The company should continue investing in high-performing product categories.

Step 16 – Region-wise Sales

```
In [19]: plt.figure(figsize=(8,5))
```

```

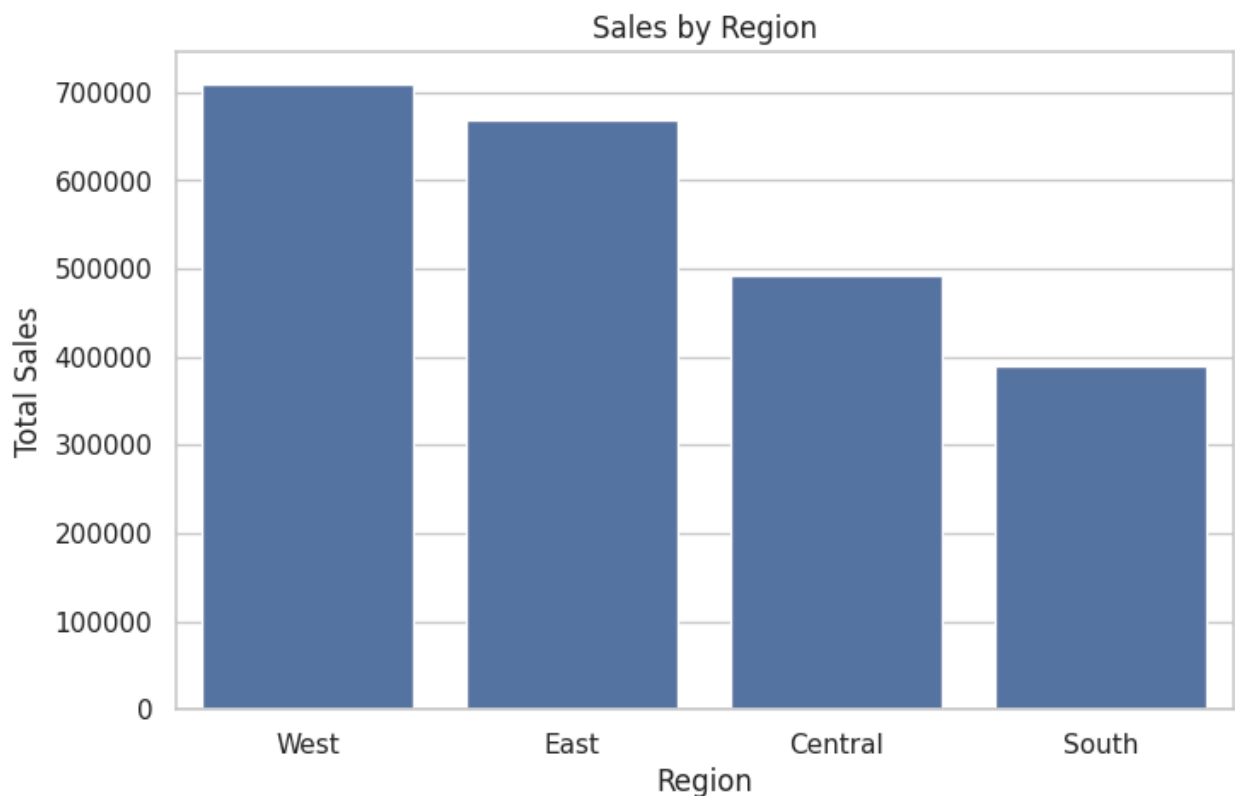
region_sales = df.groupby("Region")["Sales"].sum().sort_values(ascending=False)

sns.barplot(x=region_sales.index,
            y=region_sales.values)

plt.title("Sales by Region")
plt.xlabel("Region")
plt.ylabel("Total Sales")

plt.show()

```



Observation

- Some regions outperform others in terms of revenue.
- High-performing regions should be prioritized for expansion.
- Low-performing regions may require targeted marketing strategies.

Step 17 – Customer Segment Analysis

```

In [20]: plt.figure(figsize=(8,5))

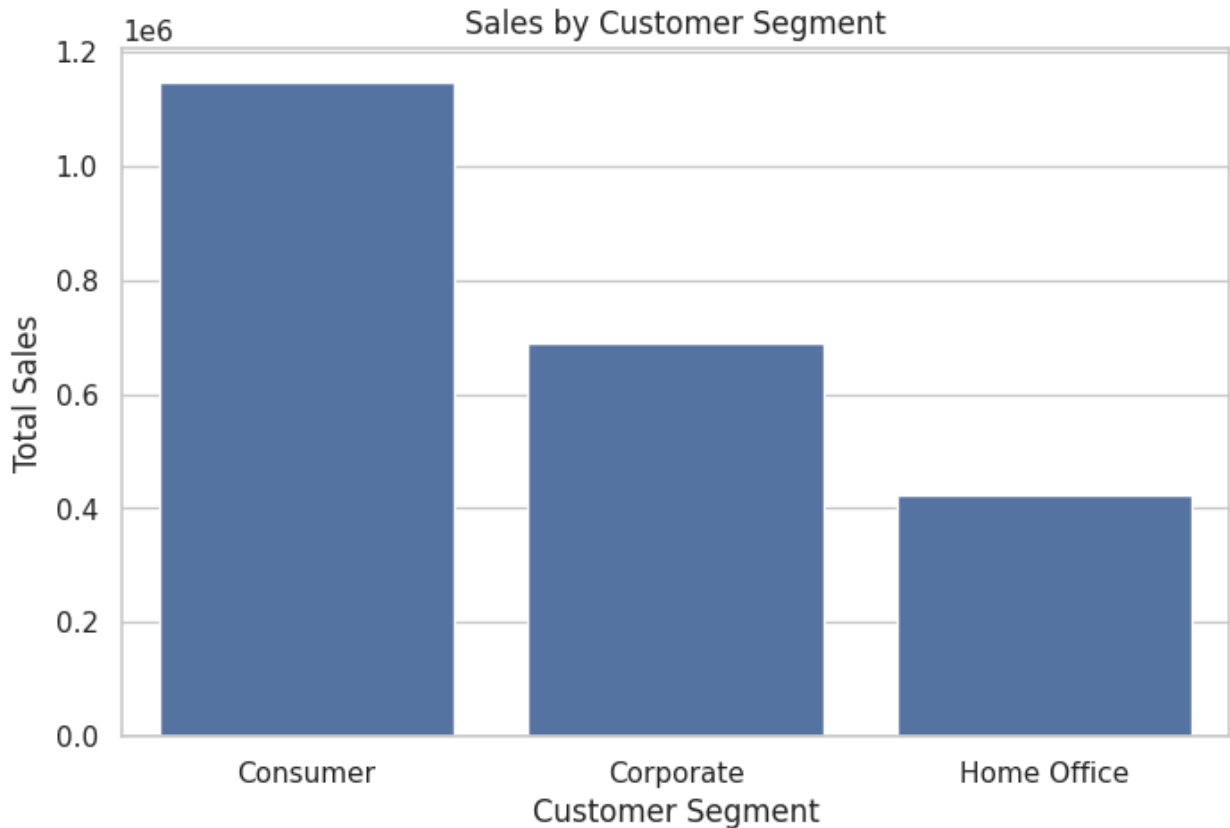
segment_sales = df.groupby("Segment")["Sales"].sum().sort_values(ascending=False)

sns.barplot(x=segment_sales.index,
            y=segment_sales.values)

```

```
plt.title("Sales by Customer Segment")
plt.xlabel("Customer Segment")
plt.ylabel("Total Sales")

plt.show()
```



Observation

- Customer segments contribute differently to total sales.
- Identifying the highest-value segment can help improve marketing campaigns.

Step 18 – Ship Mode Analysis

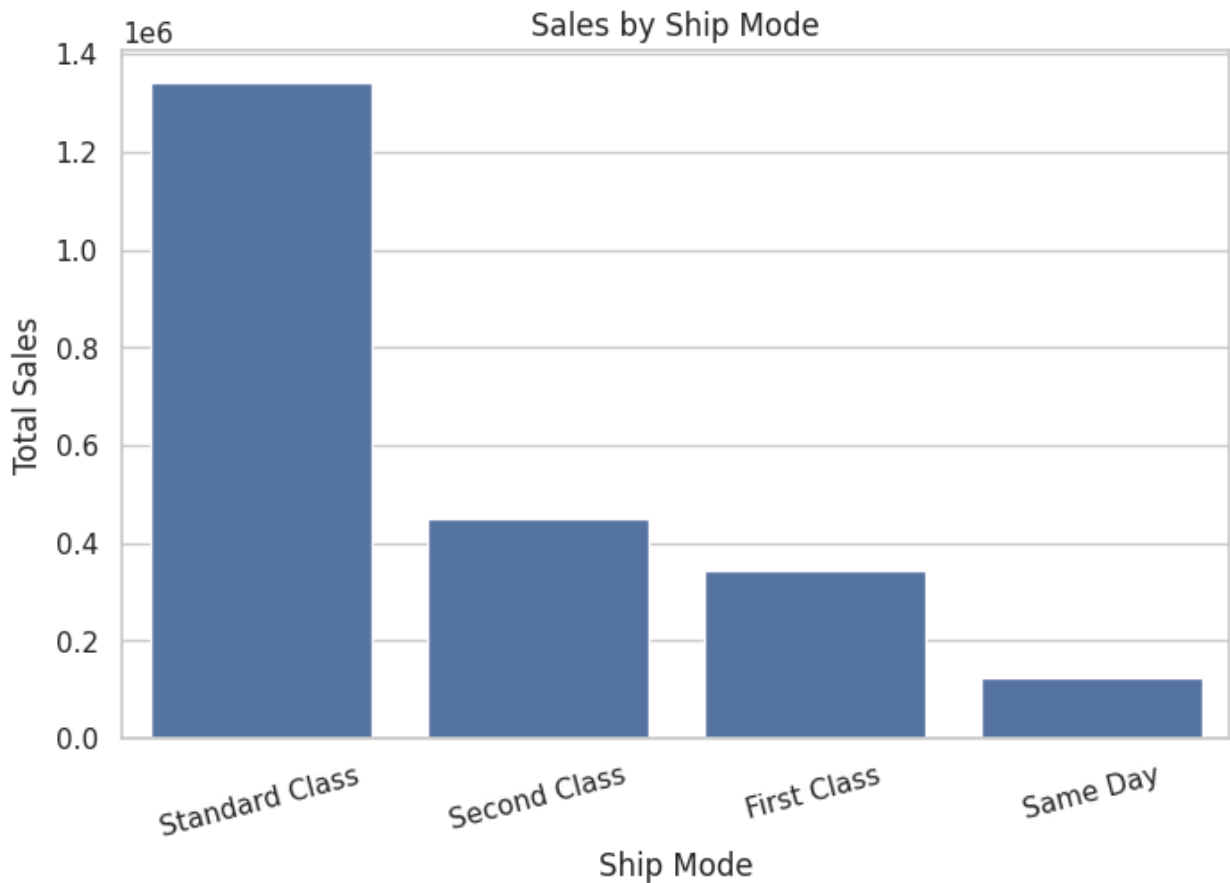
```
In [21]: plt.figure(figsize=(8,5))

ship_sales = df.groupby("Ship Mode")["Sales"].sum().sort_values(ascending=False)

sns.barplot(x=ship_sales.index,
            y=ship_sales.values)

plt.title("Sales by Ship Mode")
plt.xlabel("Ship Mode")
```

```
plt.ylabel("Total Sales")
plt.xticks(rotation=15)
plt.show()
```



Observation

- Shipping methods vary in their contribution to total sales.
- Understanding shipping preferences can improve logistics planning and customer satisfaction.

Step 19 – Top 10 Products by Sales

```
In [22]: plt.figure(figsize=(12,6))

top_products = df.groupby("Product Name")["Sales"].sum().sort_values(ascending

sns.barplot(
    x=top_products.values,
    y=top_products.index,
    palette="viridis"
```

```
)

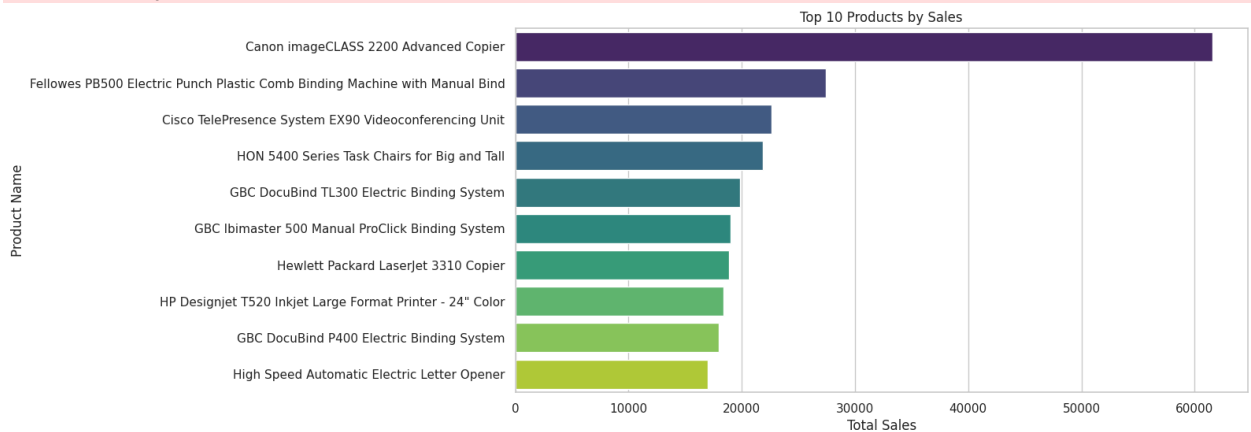
plt.title("Top 10 Products by Sales")
plt.xlabel("Total Sales")
plt.ylabel("Product Name")

plt.show()
```

/tmp/ipykernel_5362/3891227772.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(
```



Observation

- The top 10 products contribute a significant share of total revenue.
- These products should be prioritized for inventory management and promotional campaigns.

Step 20 – Monthly Sales Trend

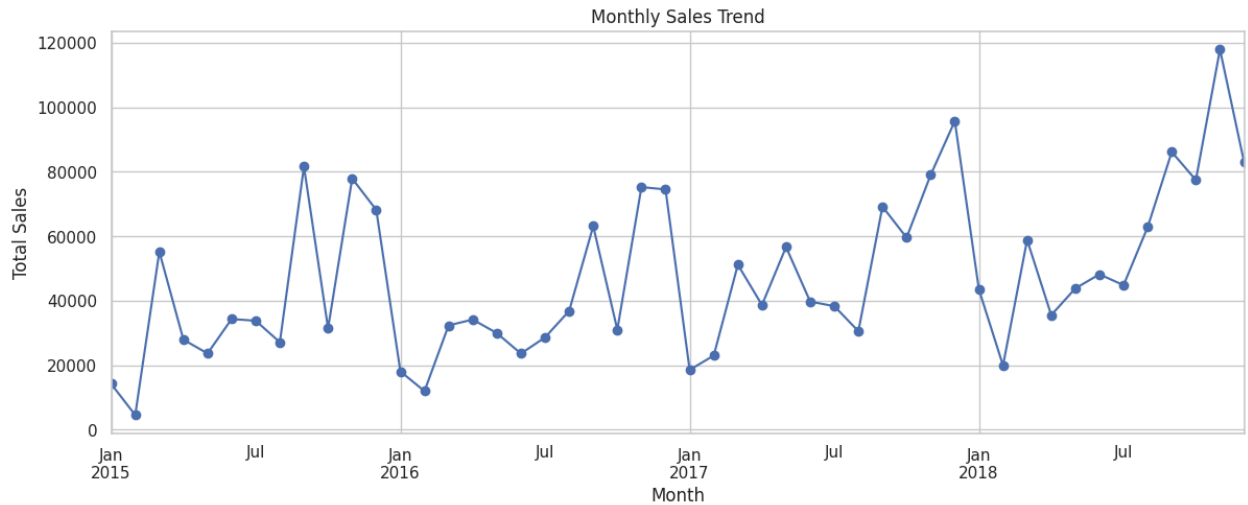
```
In [25]: df["Order Date"] = pd.to_datetime(df["Order Date"], format="%d/%m/%Y")
```

```
In [26]: monthly_sales = df.groupby(df["Order Date"].dt.to_period("M"))["Sales"].sum()

plt.figure(figsize=(14,5))
monthly_sales.plot(marker="o")

plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Total Sales")
plt.grid(True)
```

```
plt.show()
```



Observation

- Sales fluctuate throughout the year.
- Some months show peak sales, indicating seasonal demand.
- Businesses can use this information for inventory planning.

Step 21 – Quarterly Sales Trend

```
In [27]: quarterly_sales = df.groupby(df["Order Date"].dt.to_period("Q"))["Sales"].sum()

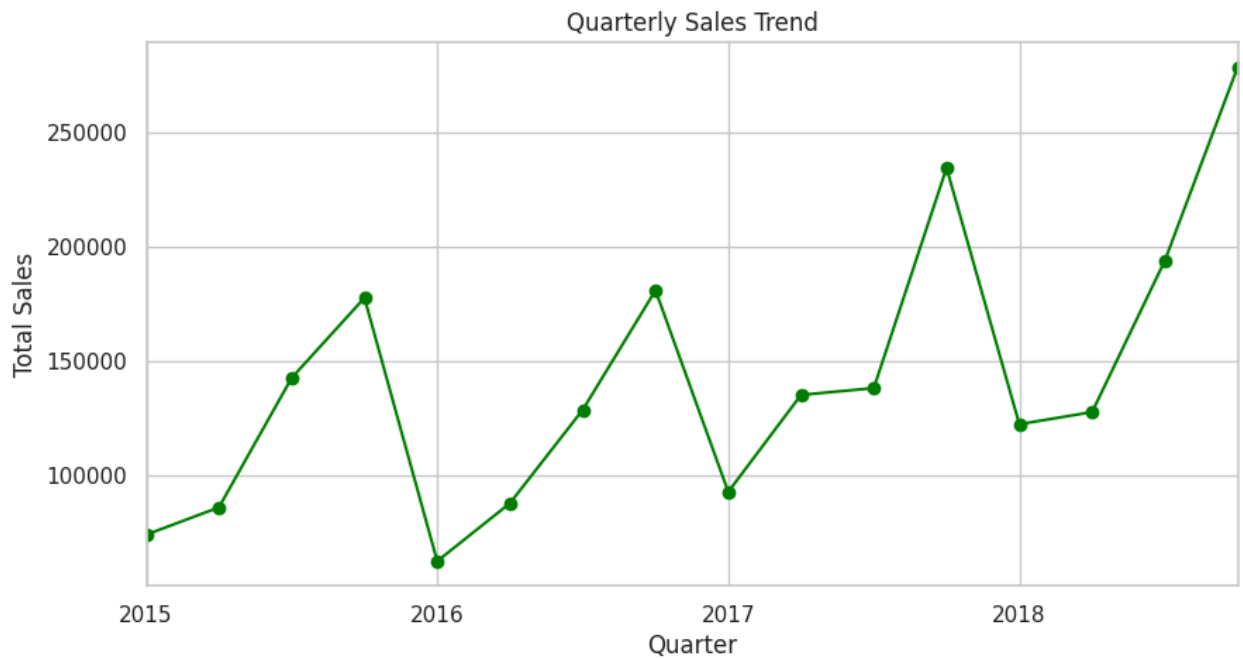
plt.figure(figsize=(10,5))

quarterly_sales.plot(marker='o', color='green')

plt.title("Quarterly Sales Trend")
plt.xlabel("Quarter")
plt.ylabel("Total Sales")

plt.grid(True)

plt.show()
```

Observation

- Quarterly analysis provides a broader view of business performance.
- It helps identify long-term sales growth and seasonal business cycles.

Step 22 – Top 10 States by Sales

```
In [28]: plt.figure(figsize=(10,6))

top_states = df.groupby("State")["Sales"].sum().sort_values(ascending=False).h

sns.barplot(
    x=top_states.values,
    y=top_states.index,
    palette="magma"
)

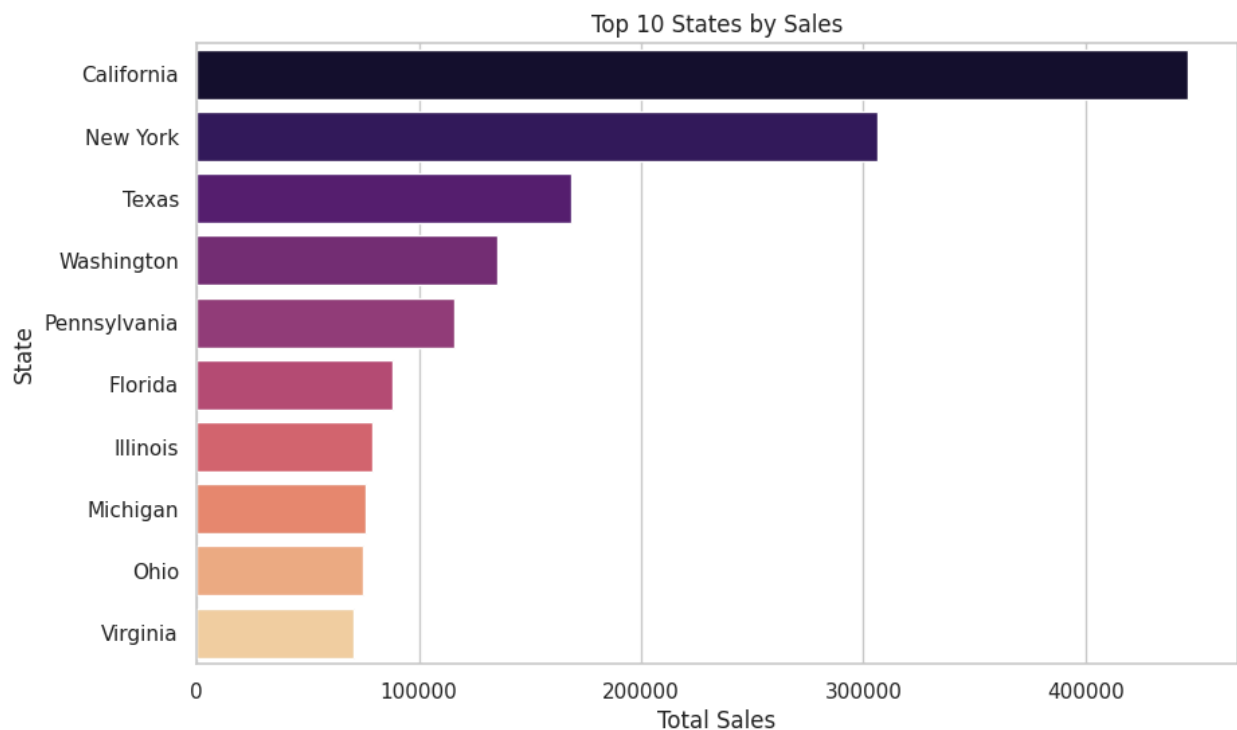
plt.title("Top 10 States by Sales")
plt.xlabel("Total Sales")
plt.ylabel("State")

plt.show()
```

/tmp/ipykernel_5362/1939749232.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(
```



Observation

- Some states generate much higher sales than others.
- High-performing states can be targeted for business expansion.

Step 23 – Correlation Heatmap

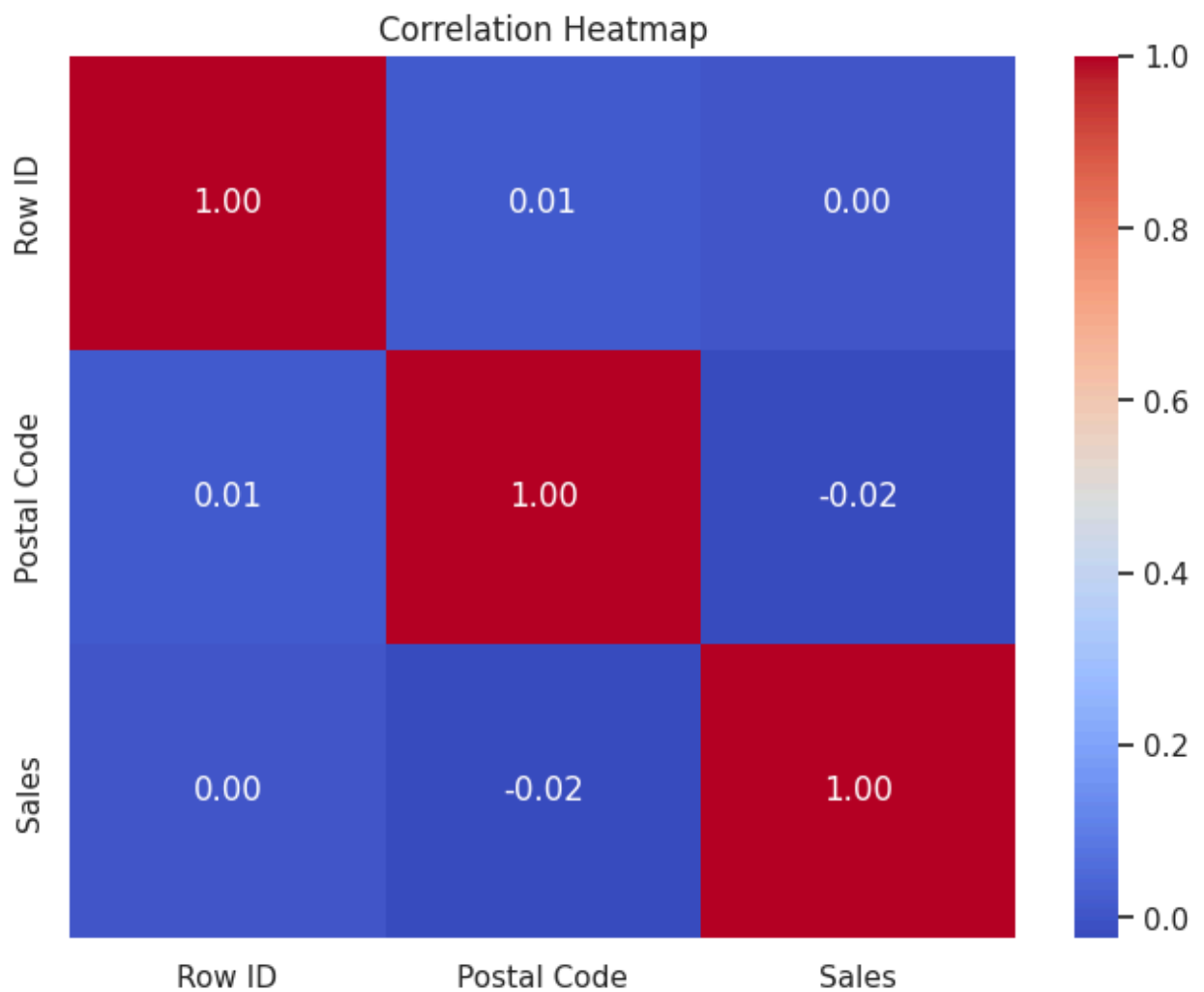
```
In [29]: plt.figure(figsize=(8,6))

corr = df.select_dtypes(include="number").corr()

sns.heatmap(
    corr,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Heatmap")

plt.show()
```



Observation

- The heatmap shows relationships among numerical variables.
- Strong positive or negative correlations help identify important business factors.

Step 24 – Executive Summary

Executive Summary

This project presents an Exploratory Data Analysis (EDA) of a retail supermarket sales dataset. The analysis focuses on understanding sales performance, customer behavior, product demand, shipping preferences, and regional trends. Various statistical techniques and visualizations were used to uncover patterns that can

help improve business decision-making.

The findings provide valuable insights into customer segments, product categories, sales trends, and operational performance. Based on these findings, several business recommendations have been proposed to support future growth and profitability.

Step 25 – Key Business Insights

Key Business Insights

1. Product Performance

Technology products generated the highest sales revenue, indicating strong customer demand.

2. Customer Segments

The Consumer segment contributed the largest share of total sales compared to Corporate and Home Office customers.

3. Regional Performance

Certain regions consistently outperformed others, highlighting opportunities for targeted marketing and expansion.

4. Sales Trend

Monthly and quarterly sales analysis revealed seasonal fluctuations, suggesting that demand changes throughout the year.

5. Shipping Analysis

Standard shipping methods handled the majority of orders, showing customer preference for cost-effective delivery.

6. Product Concentration

A small number of products generated a significant portion of overall revenue.

7. Data Quality

The dataset was successfully cleaned by handling missing values and duplicate records, ensuring reliable analysis.

Step 26 – Business Recommendations

Business Recommendations

Recommendation 1

Increase inventory levels for high-selling products to prevent stock shortages during peak demand.

Recommendation 2

Launch promotional campaigns in low-performing regions to improve market penetration.

Recommendation 3

Develop loyalty and reward programs for Consumer segment customers to encourage repeat purchases.

Recommendation 4

Improve demand forecasting using historical sales trends to optimize inventory planning.

Recommendation 5

Review shipping operations regularly to improve delivery efficiency and customer satisfaction.

Step 27 – Conclusion

Conclusion

This Exploratory Data Analysis successfully identified important sales trends, customer purchasing behavior, regional performance, and product demand within the retail supermarket dataset.

The analysis demonstrates how data analytics can support business decision-making by transforming raw data into actionable insights. The recommendations provided can help improve sales performance, operational efficiency, customer satisfaction, and long-term business growth.

Overall, this project highlights the practical application of Python, Pandas, Matplotlib, and Seaborn in solving real-world business problems through data analysis.

Step 28 – Technologies Used

Technologies Used

- Python
- Google Colab
- Pandas
- NumPy
- Matplotlib
- Seaborn

Step 29 – Future Improvements

Future Improvements

- Build an interactive Power BI dashboard.
- Develop a sales forecasting model using Machine Learning.
- Perform customer segmentation using clustering techniques.

- Integrate real-time sales data for continuous monitoring.